

MODULE 2 — CPU Scheduling

Senior SWE Interview Track — Operating Systems

2.0 The Metrics First (Waiting / Turnaround / Response Time)

1. Why Interviewers Ask This

Every scheduling question is scored with these three numbers. Interviewers watch whether you compute them correctly and, more importantly, whether you know which metric each algorithm optimizes.

2. Core Concept

For a process with **arrival time** A, **burst (service) time** B, **completion time** C, and **first-scheduled time** F: -

Turnaround time = $C - A$ (total time in system) — batch throughput metric. - **Waiting time** = Turnaround - B (time spent ready but not running). - **Response time** = $F - A$ (time until *first* CPU) — interactivity metric. Also: **throughput** (jobs/sec) and **CPU utilization**.

3. Internal Working

The scheduler runs at every timer tick, on task wakeup, and on block/exit. Its policy decides who is dispatched next from the ready queue — the policy alone determines these metrics for a fixed workload.

4. ASCII Diagram

```

arrival A          first run F                      completion C
|----wait----|====run====|--wait--|===run===|
|<----- turnaround = C - A ----->|
response = F - A    waiting = turnaround - burst

```

5. Real Production Example

Latency SLOs are response-time thinking (p99 time-to-first-byte); batch ETL pipelines optimize turnaround/throughput. A queuing web server *is* a scheduling system: request queue = ready queue.

6–9. Advantages / Trade-offs / Mistakes / Performance

- No scheduler optimizes all three at once: SJF minimizes average waiting; RR minimizes response variance at the cost of turnaround (switch overhead).
- Mistake: computing waiting time as “time before first run” — it includes *all* ready-but-not-running gaps under preemptive schedulers.
- Mistake: forgetting context-switch overhead when asked “what’s the ideal quantum”.

10–14. Interview Prep

- Q: “Given this table of arrivals/bursts, compute avg waiting time under FCFS and SJF.” (Practice by hand — asked verbatim at Amazon/Microsoft.)
- Follow-up: “Which metric matters for a web server vs a nightly batch job?”
- Practice: build a 4-process table and compute all three metrics under FCFS, SJF, RR($q=2$).

2.1 FCFS (First Come, First Served)

1. Why Interviewers Ask This

Baseline algorithm — used to test if you can spot the **convoy effect** and reason about why simple FIFO fails for latency.

2. Core Concept

Non-preemptive FIFO: run each job to completion in arrival order.

3. Internal Working

Single FIFO ready queue; dispatch head; no preemption; only blocking/exit yields the CPU.

4. ASCII Diagram

```
Arrivals: P1(burst 24), P2(3), P3(3) all at t=0
Gantt: |-----P1(24)-----|P2(3)|P3(3)|
Wait: P1=0, P2=24, P3=27 -> avg 17
Reversed order: |P2|P3|----P1----| -> avg wait (0+3+6)/3 = 3
CONVOY EFFECT: one long job makes everyone wait.
```

5. Real Production Example

A single-threaded worker consuming a FIFO queue (e.g., one Kafka partition consumer): one 30-second job starves hundreds of 10 ms jobs behind it — the convoy effect in service clothing (head-of-line blocking).

6. Advantages

Trivial, no starvation (everyone eventually runs), minimal overhead, fair in arrival order.

7. Trade-offs

Terrible average waiting time with mixed burst lengths; awful response time; convoy effect.

8. Common Mistakes

- Not naming the convoy effect.
- Saying FCFS starves jobs (it doesn't — that's priority/SJF).

9. Performance Implications

Average wait is order-dependent: schedule-short-first can cut it by an order of magnitude — the motivation for SJF.

10–11. Interview & Follow-ups

- “Compute avg wait for these arrivals.” “What is the convoy effect and where do you see it in real systems?” (HOL blocking in queues, TCP, HTTP/1.1 pipelining)

12. Coding/Debugging Scenario

Job queue p99 explodes when a “reindex-everything” task lands: split queues by expected duration or add preemption/chunking.

13. Best Practices

Separate fast/slow lanes; chunk long jobs; FCFS only where fairness-by-arrival is the actual requirement.

14. Practice Questions

1. P1..P4 arrive at 0,1,2,3 with bursts 8,4,9,5 — Gantt chart + all metrics.
2. Give two production systems where FCFS-style HOL blocking bit you and the fix.

2.2 SJF (Shortest Job First)

1. Why Interviewers Ask This

It's *provably optimal* for average waiting time — interviewers test whether you know that, and whether you spot that it's unimplementable without predicting the future.

2. Core Concept

Non-preemptive: among ready jobs, run the one with the smallest total burst. Optimal average waiting time (exchange argument: any long-before-short pair can be swapped to reduce total wait).

3. Internal Working

Ready queue = min-heap keyed by burst estimate. Real systems can't know bursts → estimate with exponential averaging: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$.

4. ASCII Diagram

```
t=0: P1(6), P2(8), P3(7), P4(3) all ready
Gantt: |P4(3)|P1(6)|P3(7)|P2(8)|
Wait: P4=0, P1=3, P3=9, P2=16 -> avg 7 (FCFS order would give 10.25)
Risk: continuous stream of short jobs -> P2 starves.
```

5. Real Production Example

Web servers doing shortest-expected-processing-first on request queues; OS I/O schedulers favoring small reads; cluster schedulers running “small jobs fast lane” (Hadoop fair-scheduler small-job preference).

6. Advantages

Minimal average waiting/turnaround time — mathematically optimal (non-preemptive, all-at-once arrivals).

7. Trade-offs

Needs burst prediction; **starvation of long jobs** under a stream of short ones; non-preemptive → bad response time when a long job is mid-run.

8. Common Mistakes

- Not mentioning starvation + aging as the fix.
- Claiming optimality without the caveat (preemption/arrivals change the picture → SRTF).

9. Performance Implications

Prediction error degrades toward FCFS behavior; heavy-tailed job sizes (common in practice) make short-first policies hugely beneficial.

10–11. Interview & Follow-ups

- “Prove/argue SJF optimality.” “How would you estimate burst length?” “How do you prevent starvation?” (aging)

12. Coding/Debugging Scenario

Implement a job runner with an exponential-average duration predictor per job type; measure avg wait vs FIFO on a heavy-tailed workload.

13. Best Practices

Use size-based scheduling where job sizes are known/predictable; always add aging or a max-wait bound.

14. Practice Questions

1. Same 4-process table: compute SJF vs FCFS average waiting time.
2. $\alpha=0.5$, $\tau_0=10$, actual bursts 6,4,6 — compute successive predictions.

2.3 SRTF (Shortest Remaining Time First)

1. Why Interviewers Ask This

The preemptive twist on SJF — tests whether you understand preemption mechanics and can re-compute schedules when arrivals interleave.

2. Core Concept

Preemptive SJF: whenever a new job arrives with remaining time shorter than the running job's remaining time, preempt. Optimal average waiting time among *all* algorithms (with known bursts).

3. Internal Working

On every arrival/completion, compare remaining times; min-heap keyed on remaining time; preemption triggers a context switch.

4. ASCII Diagram

```
P1 arr 0 burst 8 | P2 arr 1 burst 4 | P3 arr 2 burst 9 | P4 arr 3 burst 5
t: 0      1      5      10     17     26
   |P1--|P2(4)----|P4(5)|P1(rem7)|P3(9)-----|
t=1: P2(4) < P1 rem(7) -> preempt P1.
Avg wait = ((10-1-...)) : P1=9, P2=0, P3=15, P4=2 -> 6.5
```

5. Real Production Example

The idea shows up as “preempt long batch work for interactive traffic”: e.g., Linux CFS effectively favors tasks that have consumed little CPU; big-data schedulers preempt long tasks for short ad-hoc queries.

6. Advantages

Best possible average waiting/turnaround; short jobs get near-instant service.

7. Trade-offs

More context switches; worse starvation for long jobs than SJF; requires continuous remaining-time knowledge (unrealistic).

8. Common Mistakes

- Forgetting to re-evaluate at *every* arrival in hand-simulations.
- Ignoring context-switch cost when comparing to SJF.

9. Performance Implications

In heavy-tailed workloads SRTF-like policies dramatically cut mean latency, at the price of tail latency for the biggest jobs — a mean-vs-p99 trade-off interviewers love.

10–11. Interview & Follow-ups

- “Simulate this arrival table under SRTF.” “When does SRTF equal SJF?” (no arrivals mid-run) “Mean vs p99 — who wins, who loses?”

12. Coding/Debugging Scenario

Write a discrete-event simulator: jobs with Pareto-distributed sizes; compare FCFS/SJF/SRTF mean and p99 turnaround.

13. Best Practices

Use SRTF-flavored policies only with aging/priority floors so giant jobs still finish.

14. Practice Questions

1. Recompute the diagram above if P3's burst were 2.
2. Explain why SRTF minimizes mean waiting time (exchange argument with remaining times).

2.4 Priority Scheduling

1. Why Interviewers Ask This

It's how real OSes and job systems actually work, and it sets up the two must-know failure modes: **starvation** and **priority inversion**.

2. Core Concept

Each task has a priority; scheduler runs the highest-priority ready task. Variants: preemptive vs non-preemptive; static vs dynamic priorities. SJF is priority scheduling with priority = 1/burst.

3. Internal Working

- Linux: nice -20..+19 for normal tasks (CFS/EEVDF weights the *share*, not absolute order), and real-time classes `SCHED_FIFO/SCHED_RR` (priorities 1–99) that strictly preempt normal tasks.
- Ready structure: per-priority queues or a weighted virtual-runtime tree.
- **Priority inversion**: low-prio task holds a lock the high-prio task needs, while medium-prio tasks run — fixed by **priority inheritance** (lock holder temporarily inherits waiter's priority) or priority ceiling.

4. ASCII Diagram

```
Priority inversion:
H (high) --- blocked on lock L
M (medium) - RUNNING (preempts Lo)
Lo (low) --- holds lock L, never scheduled -> H waits on M!
Fix: priority inheritance -> Lo temporarily runs at H's priority.
```

5. Real Production Example

- **Mars Pathfinder (1997)**: system resets from priority inversion; fixed by enabling priority inheritance remotely — the canonical interview story.
- Kubernetes pod priority/preemption; kernel threads for interrupts at RT priority; DB systems boosting lock-holding transactions.

6. Advantages

Expresses business importance; low latency for critical work; flexible (dynamic priorities).

7. Trade-offs

Starvation of low priority; priority inversion; priority assignment becomes a config-management problem ("priority inflation" — everything ends up critical).

8. Common Mistakes

- Not knowing priority inversion or its fixes — an instant senior-level red flag.
- Assuming nice values strictly order execution on Linux (they weight CPU shares).

9. Performance Implications

A runaway `SCHED_FIFO` task can freeze a CPU (no time slicing within RT unless RR); kernel throttles RT to 95% by default (`sched_rt_runtime_us`).

10–11. Interview & Follow-ups

- "What is priority inversion? How do priority inheritance and priority ceiling differ?"

- “How does Linux nice actually affect scheduling?” “How do you stop starvation?” (aging → next section)

12. Coding/Debugging Scenario

An RT audio thread stutters: a low-prio thread holds a shared mutex. Fix:

```
pthread_mutexattr_setprotocol(PTHREAD_PRIO_INHERIT).
```

13. Best Practices

Few, well-defined priority tiers; PI mutexes when RT and normal threads share locks; never busy-loop in `SCHED_FIFO`.

14. Practice Questions

1. Tell the Mars Pathfinder story: bug, mechanism, fix.
2. Design priority tiers for a ride-hailing dispatch service (matching vs analytics vs logging).

2.5 Round Robin (RR)

1. Why Interviewers Ask This

The canonical preemptive algorithm — tests quantum-size reasoning: the tension between responsiveness and context-switch overhead.

2. Core Concept

FIFO queue + fixed time quantum q ; a task runs at most q , then is preempted to the tail. Every task gets $1/n$ of the CPU within $(n-1) \cdot q$ time — bounded response time, no starvation.

3. Internal Working

Timer interrupt at quantum expiry → preempt → enqueue at tail → dispatch head. Tasks that block before quantum end simply leave the queue (interactive tasks are naturally favored).

4. ASCII Diagram

```
q=4: P1(24), P2(3), P3(3)
|P1:4|P2:3|P3:3|P1:4|P1:4|P1:4|P1:4|P1:1|
Response: P1=0, P2=4, P3=7 (vs FCFS: 0,24,27)
q too big -> degrades to FCFS
q too small-> switch overhead dominates (e.g., q=2*switch cost -> 33% waste)
```

5. Real Production Example

- `SCHED_RR` for equal-priority real-time tasks; time-slicing in hypervisors (vCPU scheduling); token-bucket-style fair sharing among tenants; classic timesharing.
- CFS is *not* RR but inherits the “everyone gets a bounded slice” goal (targeted latency / sched_latency window).

6. Advantages

Starvation-free; predictable, bounded response time; great for interactive fairness.

7. Trade-offs

Higher turnaround for long jobs than FCFS; quantum tuning; more switches → cache pollution.

8. Common Mistakes

- No rule for choosing q . Rule of thumb: $q \gg$ context-switch cost (e.g., 100×), and sized so ~80% of interactive bursts finish within one quantum. Typical: 1–100 ms.
- Forgetting arrival-vs-requeue ordering in hand simulations (new arrival enqueues before the preempted task, in most conventions — state your convention).

9. Performance Implications

Overhead fraction $\approx \text{switch_cost} / (q + \text{switch_cost})$. $q=10 \text{ ms}$, $\text{switch}=10 \mu\text{s} \rightarrow \sim 0.1\%$ overhead; $q=100 \mu\text{s} \rightarrow \sim 9\%$.

10–11. Interview & Follow-ups

- “What happens as $q \rightarrow \infty$ and $q \rightarrow 0$?” (FCFS; processor sharing with infinite overhead)
- “Simulate RR($q=2$) for this table.” “How would you pick q for a mixed workload?”

12. Coding/Debugging Scenario

Multi-tenant worker uses RR over tenant queues; one tenant’s tasks are $100\times$ longer $\rightarrow$ apply per-quantum chunking (cooperative yield every N items).

13. Best Practices

Quantum $\geq 100\times$ switch cost; combine RR with priorities (RR within each priority level — that’s `SCHED_RR`).

14. Practice Questions

1. Bursts 5,15,4,3 at $t=0$, $q=4$ — Gantt + avg response and turnaround.
2. Derive the overhead formula and find q for $<1\%$ overhead at $5 \mu\text{s}$ switch cost.

2.6 Multilevel Queue (MLQ)

1. Why Interviewers Ask This

Tests whether you can combine algorithms into a policy: different classes of work deserve different scheduling.

2. Core Concept

Partition the ready queue into fixed queues by task class (e.g., system > interactive > batch), each with its own algorithm (RR for interactive, FCFS for batch). Between queues: strict priority or weighted CPU shares. Tasks are **permanently assigned** to a queue.

3. Internal Working

Scheduler serves the highest non-empty queue (strict) or divides bandwidth (e.g., 80/20). Strict priority $\rightarrow$ lower queues starve when upper queues stay busy.

4. ASCII Diagram

```
Q0 System/RT      [RR]      highest  --served first
Q1 Interactive    [RR q=8ms]
Q2 Batch          [FCFS]     lowest   --runs only when Q0,Q1 empty
Task class fixed at creation -> misclassification is permanent.
```

5. Real Production Example

Linux scheduling classes are exactly this: `stop` > `deadline` > `rt` (FIFO/RR) > `fair` (CFS/EEVDF) > `idle`.
Kubernetes QoS classes (Guaranteed/Burstable/BestEffort) are MLQ thinking at the cluster level.

6. Advantages

Simple; strong guarantees for the top class; per-class tailored policy.

7. Trade-offs

Rigid — no migration between queues; starvation of low queues; misclassification is unfixable at runtime.

8. Common Mistakes

- Confusing MLQ with MLFQ (the *feedback*/migration is the whole difference).
- Not mentioning starvation under strict inter-queue priority.

9. Performance Implications

Strict-priority MLQ gives excellent latency for the top tier and unbounded tail latency for the bottom tier — say this explicitly.

10–11. Interview & Follow-ups

- “MLQ vs MLFQ?” “How does Linux layer its scheduling classes?” “Fixed priority between queues vs time-slicing between queues?”

12. Coding/Debugging Scenario

Batch jobs never run during business hours in a strict-MLQ job system → add a bandwidth share (e.g., batch guaranteed 10%).

13. Best Practices

Give every queue a minimum bandwidth; keep classes few and observable.

14. Practice Questions

1. Design MLQ tiers for a database server (replication, queries, vacuum/compaction).
2. When is strict priority between queues actually correct? (hard real-time)

2.7 Multilevel Feedback Queue (MLFQ)

1. Why Interviewers Ask This

The most-asked “design a scheduler” answer: it *learns* job behavior without prior knowledge — and it lets interviewers probe starvation, aging, and gaming.

2. Core Concept

Multiple queues by priority; **tasks move between queues based on observed behavior**: use your full quantum (CPU-bound) → demoted; block quickly (interactive/I/O-bound) → stay high or get promoted. Approximates SRTF with zero foreknowledge.

3. Internal Working

Classic rules: 1. Higher queue runs first; RR within a queue. 2. New tasks enter the top queue. 3. Use entire quantum (cumulative, to prevent gaming) → drop one level; longer quanta at lower levels. 4. **Aging/priority boost**: periodically move everyone to the top queue → prevents starvation and adapts to phase changes.

4. ASCII Diagram

```

Q0 (q=8ms) [I/O-bound stay here] <---- periodic boost (aging)
  | used full quantum                ^
  v                                |
Q1 (q=16ms)                        |
  | used full quantum                |
  v                                |
Q2 (q=64ms, FCFS-ish) [CPU hogs end here] --+

```

5. Real Production Example

Windows scheduler and Solaris TS class are MLFQ-flavored (priority decay + boosts, e.g., foreground-window and I/O-completion boosts). Historical BSD/ULE similarly. Linux CFS took a different route (fair virtual runtime) but achieves the same “interactive tasks feel snappy” goal — a great compare-and-contrast to volunteer.

6. Advantages

No burst prediction needed; excellent interactive latency; adapts to behavior changes; starvation-free *with* boosting.

7. Trade-offs

Many knobs (levels, quanta, boost period); **gameable** (yield just before quantum end to stay high — fixed by charging cumulative CPU time); boosts cause periodic latency blips for the top queue.

8. Common Mistakes

- Omitting the priority boost → your design starves batch jobs; interviewers *will* ask.
- Not addressing gaming.

9. Performance Implications

Approximates SRTF's mean-latency win for heavy-tailed workloads; the boost period trades starvation bound vs interactive purity.

10–11. Interview & Follow-ups

- “Design a scheduler that needs no knowledge of job lengths.” (this)
- “How can a process game MLFQ, and how do you fix it?” “What does the boost period control?”
- “Compare MLFQ with CFS's virtual-runtime approach.”

12. Coding/Debugging Scenario

Implement MLFQ in a request-processing framework: requests demoted after N ms of service; measure short-request p50 improvement vs FIFO.

13. Best Practices

Charge cumulative CPU; boost on a period $\approx$ seconds; expose queue-level metrics.

14. Practice Questions

1. Simulate: A(CPU-bound, 200ms) and B(1ms CPU + 10ms I/O loop) through a 3-level MLFQ.
2. Choose quanta and boost period for a mixed API+batch worker; justify.

2.8 Starvation & Aging

1. Why Interviewers Ask This

The standard follow-up to *every* scheduling answer: “what breaks, and how do you fix it?”

2. Core Concept

Starvation: a runnable task waits unboundedly because the policy always prefers others (SJF/SRTF long jobs, low priorities, readers-vs-writers). **Aging**: gradually increase the priority of waiting tasks so waiting time itself becomes priority — converts unbounded starvation into a bounded delay.

3. Internal Working

- Implementations: priority $\pm$ f(wait time) recomputed on a timer; MLFQ boost; CFS-style vruntime (waiting tasks accumulate less vruntime → become “owed” CPU — aging built into the algorithm).
- Same idea beyond CPU: lock fairness (ticket locks), DB lock queues (FIFO grant), network schedulers (deficit round robin).

4. ASCII Diagram

```

prio
^
|      aging:  $p(t) = p_0 + k \cdot \text{wait}$ 
|      /----- low-prio task finally exceeds
|      -----/      competing tasks and runs
|_____/
+-----> wait time
Starvation = wait unbounded; aging bounds it.

```

5. Real Production Example

- Message consumers where “hot” partitions always win → cold partitions starve; fix with deficit-weighted polling.
- Writer starvation with reader-preferring RW locks (Module 3).
- Linux RT throttling (95%) = built-in anti-starvation for normal tasks under runaway RT threads.

6–7. Advantages / Trade-offs

Aging guarantees progress; but weakens the intended priority discipline and adds tuning (rate k). Too-fast aging $\approx$ FIFO; too-slow $\approx$ starvation anyway.

8. Common Mistakes

- Confusing starvation with deadlock (starved task *can* run if given CPU; deadlocked can’t ever).
- Proposing aging without a concrete mechanism.

9. Performance Implications

Anti-starvation mechanisms cost tail latency of top-priority work — quantify it (boost period, aging rate).

10–11. Interview & Follow-ups

- “Which of the algorithms we discussed can starve, and why?” (SJF, SRTF, strict priority, strict MLQ; not FCFS/RR)
- “Starvation vs deadlock vs livelock?”

12. Coding/Debugging Scenario

Low-priority reconciliation loop never runs during traffic peaks → add a minimum-share guarantee or aging to the internal scheduler.

13. Best Practices

Every prioritized system needs an explicit starvation story: aging, minimum shares, or FIFO fallback; monitor max wait per class.

14. Practice Questions

1. Identify starvation risk in: SJF, RR, strict MLQ, reader-preferring RW lock — and the fix for each.
2. Design aging for a support-ticket queue with P0–P3 severities.

Module 2 Cheat Sheet (one page)

Algorithm	Preemptive?	Optimizes	Starvation?	Killer fact
FCFS	No	Simplicity	No	Convoy effect
SJF	No	Avg waiting (optimal)	Yes (long jobs)	Needs burst prediction (exp. averaging)
SRTF	Yes	Avg waiting (optimal overall)	Yes (worse)	Re-evaluate at every arrival
Priority	Either	Business importance	Yes	Priority inversion → inheritance (Mars Pathfinder)
RR	Yes	Response time	No	$q \gg$ switch cost; $q \rightarrow \infty =$ FCFS
MLQ	Yes	Per-class policy	Yes (low queues)	Fixed assignment; Linux sched classes
MLFQ	Yes	SRTF w/o foreknowledge	No (with boost)	Demote CPU hogs; periodic boost; anti-gaming

Metrics: turnaround = $C - A$; waiting = turnaround – burst; response = first-run – A. **Formulas:** RR overhead $\approx s/(q+s)$; exp. average $\tau' = \alpha t + (1-\alpha)\tau$. **Linux reality:** classes stop>deadline>rt>fair(CFS/EEVDF)>idle; nice = weight not order; RT throttled at 95%.

Top Interview Questions

1. Compute waiting/turnaround for a table under FCFS, SJF, SRTF, RR (do these by hand until fast).
2. What is the convoy effect? Where does it appear outside CPU scheduling?
3. Priority inversion: mechanism, Mars Pathfinder, priority inheritance vs ceiling.
4. Design a scheduler with unknown job lengths → MLFQ, with gaming + starvation defenses.
5. How does Linux actually schedule (CFS/EEVDF, nice weights, RT classes)?
6. How do you pick an RR quantum?
7. Starvation vs deadlock vs livelock.

Common Mistakes (module-wide)

- Metric arithmetic slips (waiting vs response); not stating tie-break conventions.
- SJF/SRTF answers without starvation+aging; MLFQ without boost or anti-gaming.
- Ignoring context-switch overhead; treating nice as strict ordering.
- Confusing MLQ (fixed) with MLFQ (feedback).

Mock Interview (self-test, ~20 min)

1. (Compute) P1..P4 arrive 0,2,4,5, bursts 7,4,1,4: Gantt + avg waiting under SRTF and RR($q=2$).
2. (Design) Build the scheduler for a multi-tenant CI system: interactive PR builds + nightly full builds. Policy, starvation defense, metrics.
3. (Depth) Your MLFQ demotes a video-encoding job to the bottom queue, but it's the paying customer's job. Reconcile priorities with feedback scheduling.
4. (Trap) "SJF is optimal, so why doesn't Linux use it?" (prediction, starvation, interactivity, fairness)
5. (Prod) On a pinned 4-core VM, one **SCHED_FIFO** thread spins at 100%. What happens to normal tasks, and what saves them?